

Atividade Guiada: Automação de Testes com PHPUnit

O objetivo desta atividade é instalar, configurar e executar o seu primeiro teste unitário automatizado utilizando o framework PHPUnit num sistema operativo Windows.

Pré-requisitos no Windows

Antes de começar, certifique-se de que tem o **PHP** e o **Composer** instalados globalmente e configurados nas Variáveis de Ambiente do seu Windows (PATH).

- Para verificar se estão prontos, abra o Prompt de Comando (cmd) ou PowerShell e digite:

```
CMD
```

```
php -v
```

```
composer --version
```

Se ambos os comandos retornarem às respectivas versões, o seu ambiente está pronto.

Passo 1: Criação do Diretório do Projeto

Vamos criar a pasta do projeto utilizando os comandos nativos do terminal do Windows.

1. Abra o seu terminal (CMD ou PowerShell).
2. Execute os seguintes comandos para criar e entrar na nova pasta:

```
CMD
```

```
mkdir %USERPROFILE%\Desktop\atividade-phpunit
```

```
cd %USERPROFILE%\Desktop\atividade-phpunit
```

(Este comando irá criar a pasta diretamente no seu Ambiente de Trabalho/Área de Trabalho).

3. Abra esta pasta no seu editor de código (ex: VS Code):

```
CMD
```

```
code .
```

Passo 2: Inicialização do Composer e Instalação do PHPUnit

No Windows, o Composer criará uma pasta chamada vendor e os arquivos executáveis adaptados para o ecossistema Windows (arquivos .bat e .ps1) dentro de vendor\bin\.

1. No terminal do VS Code ou no CMD/PowerShell dentro da pasta do projeto, inicialize o Composer:

```
CMD
```

```
composer init
```

*Pode pressionar **ENTER** para todas as perguntas para aceitar as configurações padrão.*

2. Instale o PHPUnit como dependência de desenvolvimento:

```
CMD
```

```
composer require --dev phpunit/phpunit
```

Aguarde a conclusão do download. O Composer criará a estrutura de pastas e o arquivo composer.json.

Passo 3: Criação da Estrutura de Pastas e Código de Negócio

No ecossistema Windows, utilizamos a barra invertida (\) para caminhos de arquivos no terminal. Vamos criar as pastas src (para o código-fonte) e tests (para os testes).

1. Crie as pastas necessárias executando:

```
CMD
```

```
mkdir src
mkdir tests
```

2. Dentro da pasta src, crie o arquivo Calculadora.php e adicione o seguinte código de negócio:

PHP

```
<?php
```

```
// src/Calculadora.php
```

```
class Calculadora {
    public function adicionar float $a float $b float {
        return $a + $b;
    }
}
```

Passo 4: Configuração do Autoload no composer.json

Para que o PHPUnit consiga encontrar a classe Calculadora sem que tenhamos de fazer require manuais, precisamos de configurar o mapeamento de classes.

1. Abra o arquivo composer.json que foi gerado na raiz do seu projeto.
2. Atualize o arquivo para incluir a seção "autoload". Ele deve ficar assim:

JSON

```
{
    "require-dev": {
        "phpunit/phpunit": "^10.0"
    },
    "autoload": {
        "classmap": [
            "src/"
        ]
    }
}
```

3. No terminal, execute o comando abaixo para que o Composer leia a nova configuração e mapeie a pasta src\:

CMD

```
composer dump-autoload
```

Passo 5: Configuração Global do PHPUnit (phpunit.xml)

No Windows, precisamos explicitamente de dizer ao PHPUnit para ativar o suporte a cores no terminal, caso contrário o resultado será exibido apenas em texto a preto e branco.

1. Na raiz do seu projeto, crie um arquivo chamado phpunit.xml.
2. Insira a seguinte estrutura de configuração:

XML

```
<?xml version="1.0" encoding="UTF-8"?>
<phpunit bootstrap "vendor/autoload.php" colors "true"
testsuites
    <testsuite name "Suite de Testes da Aplicacao"
```

```
directory suffix "Test.php" ./tests directory
testsuite
testsuites
phpunit
```

Passo 6: Criação do Teste Unitário (Padrão Triple A)

Agora vamos criar o script que validará o comportamento da nossa calculadora. O nome do arquivo deve obrigatoriamente terminar com Test.php.

1. Dentro da pasta tests\, crie o arquivo CalculadoraTest.php.
2. Escreva o código seguindo rigorosamente a estrutura de herança do PHPUnit e as três fases do **Triple A (Arrange, Act, Assert)**:

PHP

```
<?php
```

```
// tests/CalculadoraTest.php
```

```
use PHPUnit\Framework\TestCase;
```

```
class CalculadoraTest extends TestCase {
```

```
    public function testAdicionarDoisNumerosPositivos() void {
```

```
        // 1. Arrange (Preparação do cenário)
```

```
        $calculadora = new Calculadora();
```

```
        $num1 = 10.0;
```

```
        $num2 = 5.0;
```

```
        $resultadoEsperado = 15.0;
```

```
        // 2. Act (Ação / Execução do método)
```

```
        $resultadoObtido = $calculadora->adicionar($num1, $num2);
```

```
        // 3. Assert (Verificação / Asserção)
```

```
        $this->assertEquals($resultadoEsperado, $resultadoObtido);
```

```
    }
```

```
}
```

Passo 7: Execução dos Testes no Windows

A execução de binários locais no Windows varia dependendo do terminal que está a utilizar devido às políticas de segurança e sintaxe. Escolha o comando adequado para o seu caso:

- Se estiver a usar o Prompt de Comando (CMD) tradicional:

CMD

```
vendor\bin\phpunit
```

- Se estiver a usar o PowerShell (ou o terminal padrão do VS Code no Windows):

PowerShell

```
.\vendor\bin\phpunit
```

- Alternativa Universal (Evita erros de permissão/política de execução no Windows):

CMD

php vendor\bin\phpunit

Análise do Feedback Visual no Windows:

- **Cenário de Sucesso:** Se tudo estiver correto, o terminal do Windows exibirá um **ponto verde (.)**, indicando que o teste passou com sucesso.
- **Cenário de Falha (Simulação):** Abra o arquivo tests\CalculadoraTest.php, mude o \$resultadoEsperado para 20.0 e execute novamente o comando no terminal.

O terminal do Windows exibirá um **F vermelho**, detalhando exatamente a falha:

Failed asserting that 15.0 matches expected 20.0.

(Desta forma, os alunos conseguem compreender visualmente como a ferramenta acusa um erro na aplicação).